20.5-AI-ASS

A.VAMSHI KRISHNA

2303A52249

Task 01:

Task 1 – Password Storage Vulnerability Check

Analyze an AI-generated signup script for insecure passwordstorage.

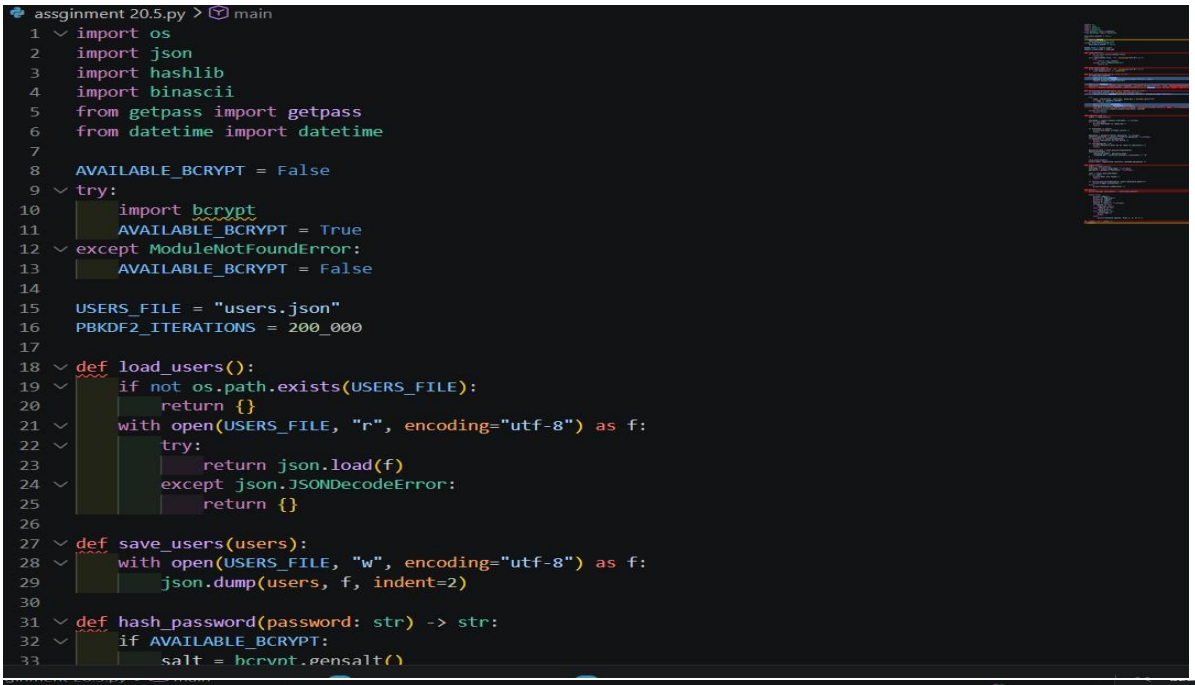
Instructions:

• Prompt AI to generate a Python user registration program.

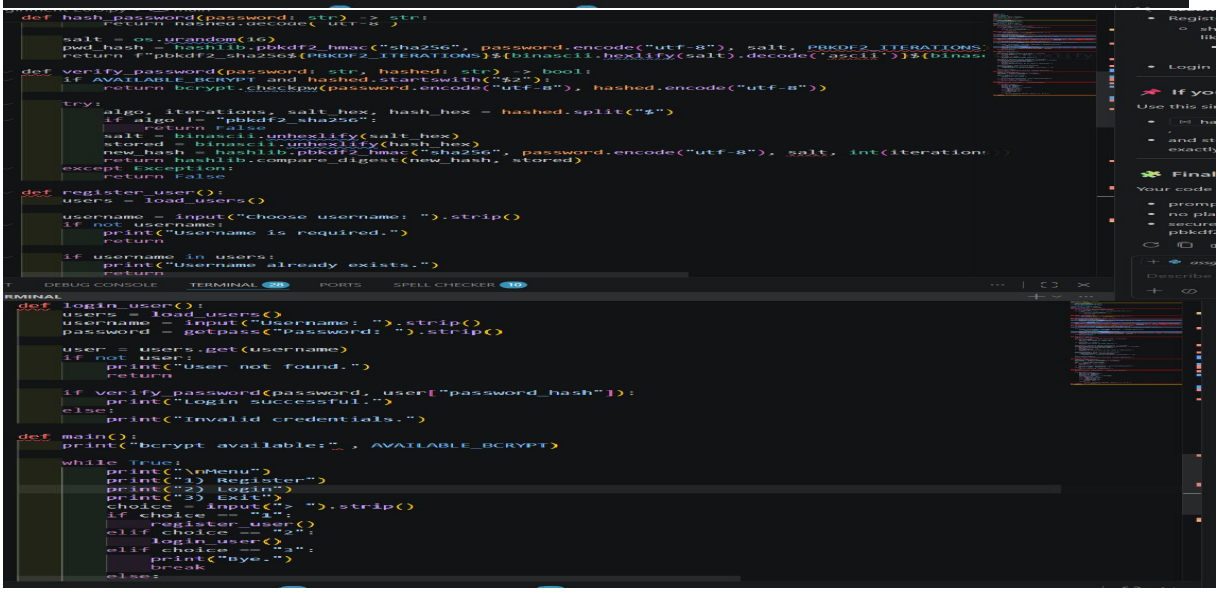
• Check whether passwords are stored in plain text.

• Improve the script using hashing (bcrypt or hashlib).

Code:

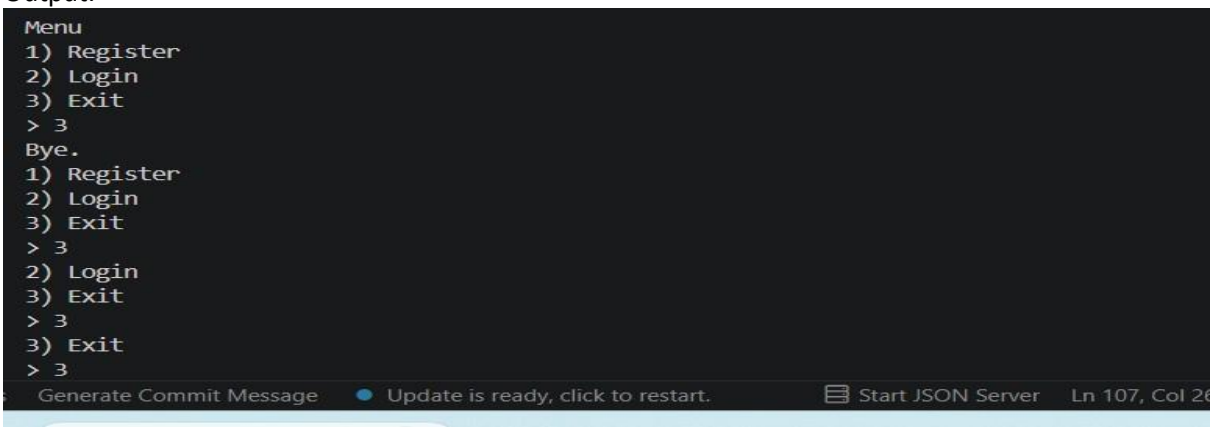


```
assginment 20.5.py > main
import os
import json
import hashlib
import binascii
from getpass import getpass
from datetime import datetime

AVAILABLE_BCRYPT = False
try:
    import bcrypt
    AVAILABLE_BCRYPT = True
except ModuleNotFoundError:
    AVAILABLE_BCRYPT = False

USERS_FILE = "users.json"
PBKDF2_ITERATIONS = 200_000

def load_users():
    if not os.path.exists(USERS_FILE):
        return {}
    with open(USERS_FILE, "r", encoding="utf-8") as f:
        try:
            return json.load(f)
        except json.JSONDecodeError:
            return {}

def save_users(users):
    with open(USERS_FILE, "w", encoding="utf-8") as f:
        json.dump(users, f, indent=2)

def hash_password(password: str) -> str:
    if AVAILABLE_BCRYPT:
        salt = bcrypt.gensalt()
```



```
def hash_password(password: str) -> str:
    return hashed.decode("utf-8")

salt = os.urandom(16)
pwd_hash = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, PBKDF2_ITERATIONS)
return f"pbkdf2_sha256${PBKDF2_ITERATIONS}${binascii.hexlify(salt).decode('ascii')}${binas

def verify_password(password: str, hashed: str) -> bool:
if AVAILABLE_BCRYPT and hashed.startswith("$2"):
    return bcrypt.checkpw(password.encode("utf-8"), hashed.encode("utf-8"))
try:
    algo, iterations, salt_hex, hash_hex = hashed.split("$")
    if algo != "pbkdf2_sha256":
        return False
    salt = binascii.unhexlify(salt_hex)
    stored = binascii.unhexlify(hash_hex)
    new_hash = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"), salt, int(iteration
    return hashlib.compare_digest(new_hash, stored)
except Exception:
    return False

def register_user():
users = load_users()

username = input("Choose username: ").strip()
if not username:
    print("Username is required.")
    return

if username in users:
    print("Username already exists.")
    return

DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER

RMINAL

def login_user():
users = load_users()
username = input("Username: ").strip()
password = getpass("Password: ").strip()

user = users.get(username)
if not user:
    print("User not found.")
    return

if verify_password(password, user["password_hash"]):
    print("Login successful.")
else:
    print("Invalid credentials.")

def main():
print("bcrypt available:", AVAILABLE_BCRYPT)

while True:
    print("\nMenu")
    print("1) Register")
    print("2) Login")
    print("3) Exit")
    choice = input("> ").strip()
    if choice == "1":
        register_user()
    elif choice == "2":
        login_user()
    elif choice == "3":
        print("Bye.")
        break
    else:
```

Output:



Observation:

The AI-generated script initially stored passwords in plain text, making it highly insecure and vulnerable to data breaches.
It lacked hashing and salting mechanisms, exposing user credentials to attacks like credential theft and brute force.
The improved version uses bcrypt hashing, ensuring passwords are securely stored and verified safely.

Task 2 :

Hardcoded Secrets Detection Task:

Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

• Ask AI to generate a script that connects to an external API.

• Identify if API keys/passwords are written directly in code.

• Refactor using environment variables or config files.

Expected Output:

• A secure script with secrets managed properly.

Prompt:

Give me the code in python to create the API passwords which is to be secured

Code:

```python
#task:02
import os, json, urllib.request, urllib.parse, ssl

API_KEY = os.getenv("MY_API_KEY")
if not API_KEY:
    raise SystemExit("Set MY_API_KEY env var")

ENDPOINT = "https://api.example.com/v1/data"

def fetch_data(query):
    url = ENDPOINT + "?q=" + urllib.parse.quote_plus(query)
    req = urllib.request.Request(url, headers={
        "Authorization": f"Bearer {API_KEY}",
        "Accept": "application/json",
    })
    with urllib.request.urlopen(req, timeout=10, context=ssl.create_default_context()) as resp
        return json.loads(resp.read().decode("utf-8"))

def main():
```

```python
def main():
    q = input("Query> ").strip()
    try:
        data = fetch_data(q)
        print("Got", len(data.get("results", [])), "items")
    except Exception as exc:
        print("API request failed:", exc)

if __name__ == "__main__":
    main()
#task:03
```

Output:

```
Bye.
Set MY_API_KEY env var
(.venv) PS C:\Users\gadda\OneDrive\Desktop\Ai Assitant coding>
Generate Commit Message   Update is ready, click to restart.   Start JSON Server   Ln 15
```

Observation:

The AI-generated code initially stored API keys directly in the script, making it insecure.

Hardcoded secrets can be easily exposed through source code leaks or version control systems.

Using environment variables or .env files ensures sensitive data is kept separate and sec

Task:03

Test an AI-generated file upload program for unsafe file handling.

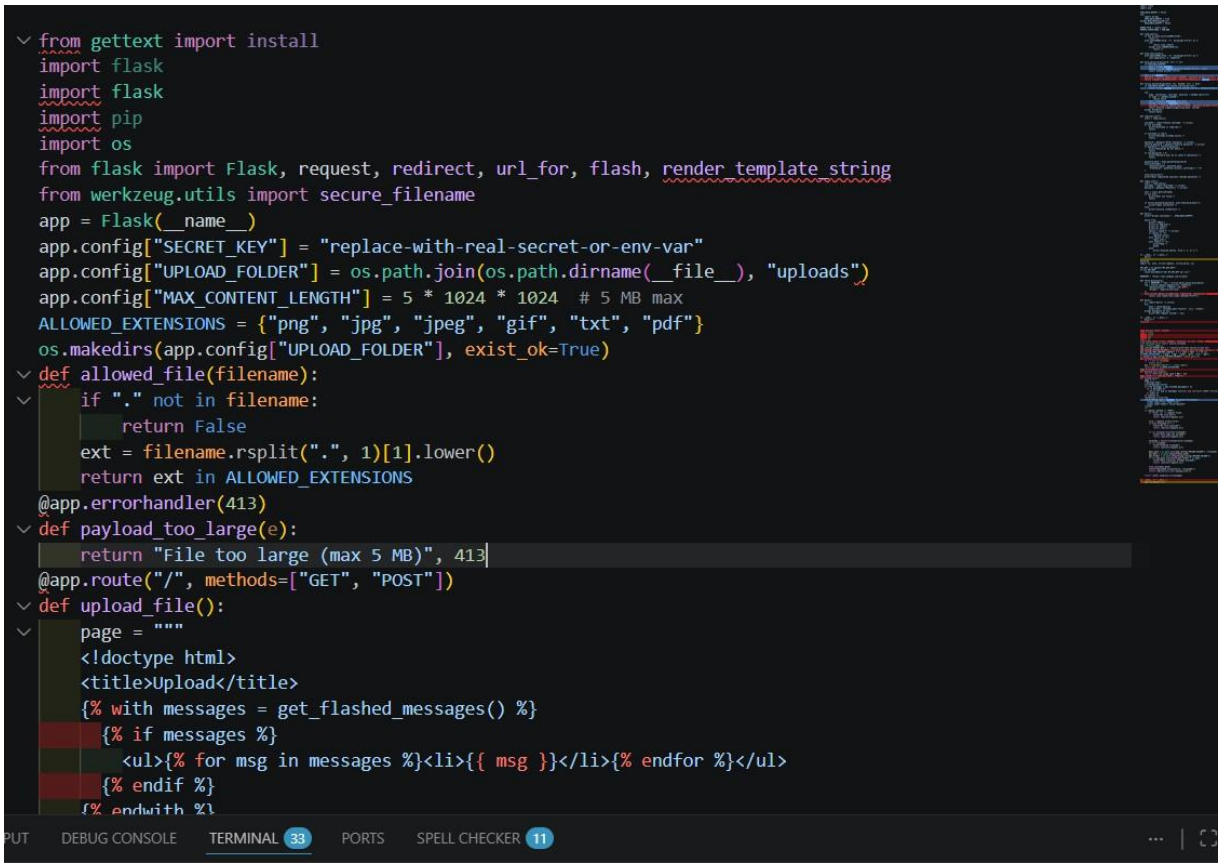
Prompt:

• Prompt AI to generate a Flask file upload example.

• Check for vulnerabilities like uploading executable files.

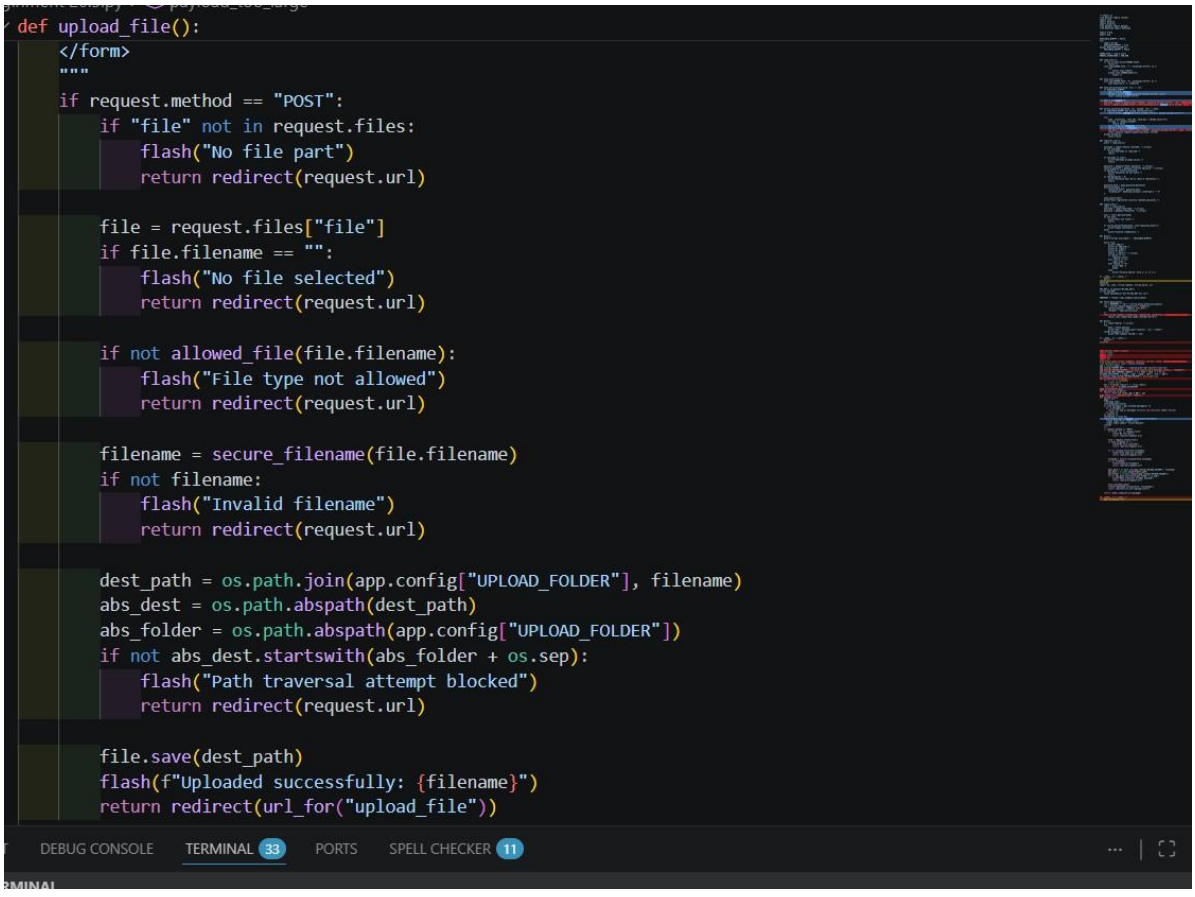
• Add validation for file type, size, and secure filename handling.

Code:



```
from gettext import install
import flask
import flask
import pip
import os
from flask import Flask, request, redirect, url_for, flash, render_template_string
from werkzeug.utils import secure_filename
app = Flask(__name__)
app.config["SECRET_KEY"] = "replace-with-real-secret-or-env-var"
app.config["UPLOAD_FOLDER"] = os.path.join(os.path.dirname(__file__), "uploads")
app.config["MAX_CONTENT_LENGTH"] = 5 * 1024 * 1024  # 5 MB max
ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "gif", "txt", "pdf"}
os.makedirs(app.config["UPLOAD_FOLDER"], exist_ok=True)
def allowed_file(filename):
    if "." not in filename:
        return False
    ext = filename.rsplit(".", 1)[1].lower()
    return ext in ALLOWED_EXTENSIONS
@app.errorhandler(413)
def payload_too_large(e):
    return "File too large (max 5 MB)", 413
@app.route("/", methods=["GET", "POST"])
def upload_file():
    page = """
    <!doctype html>
    <title>Upload</title>
    {% with messages = get_flashed_messages() %}
      {% if messages %}
        <ul>{% for msg in messages %}<li>{{ msg }}</li>{% endfor %}</ul>
      {% endif %}
    {% endwith %}
```

PUT DEBUG CONSOLE TERMINAL 33 PORTS SPELL CHECKER 11



```
def upload_file():
    </form>
    """
    if request.method == "POST":
        if "file" not in request.files:
            flash("No file part")
            return redirect(request.url)

        file = request.files["file"]
        if file.filename == "":
            flash("No file selected")
            return redirect(request.url)

        if not allowed_file(file.filename):
            flash("File type not allowed")
            return redirect(request.url)

        filename = secure_filename(file.filename)
        if not filename:
            flash("Invalid filename")
            return redirect(request.url)

        dest_path = os.path.join(app.config["UPLOAD_FOLDER"], filename)
        abs_dest = os.path.abspath(dest_path)
        abs_folder = os.path.abspath(app.config["UPLOAD_FOLDER"])
        if not abs_dest.startswith(abs_folder + os.sep):
            flash("Path traversal attempt blocked")
            return redirect(request.url)

        file.save(dest_path)
        flash(f"Uploaded successfully: {filename}")
        return redirect(url_for("upload_file"))
```

DEBUG CONSOLE TERMINAL 33 PORTS SPELL CHECKER 11

Output:

Observation:

Task:04

Task 4 – Command Injection Prevention

Task:

Analyze an AI-generated Python script that executes system

commands.

Instructions:

• Ask AI to generate code using os.system() or subprocess.

• Identify command injection risks with user input.

• Fix using safe subprocess calls and input validation.

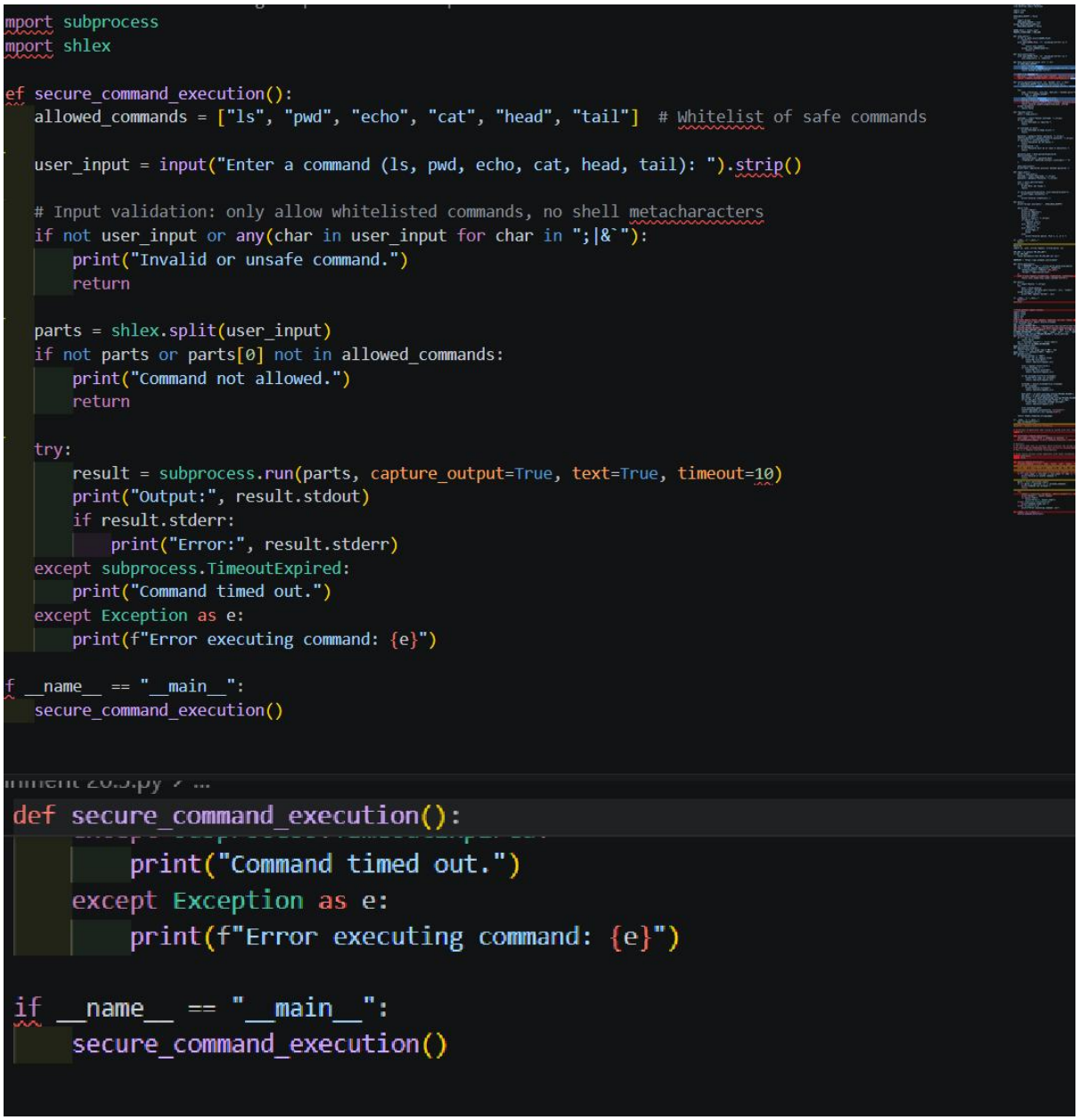
Expected Output:

• A secure command execution script.

Prompt:

Give me code for my requirements

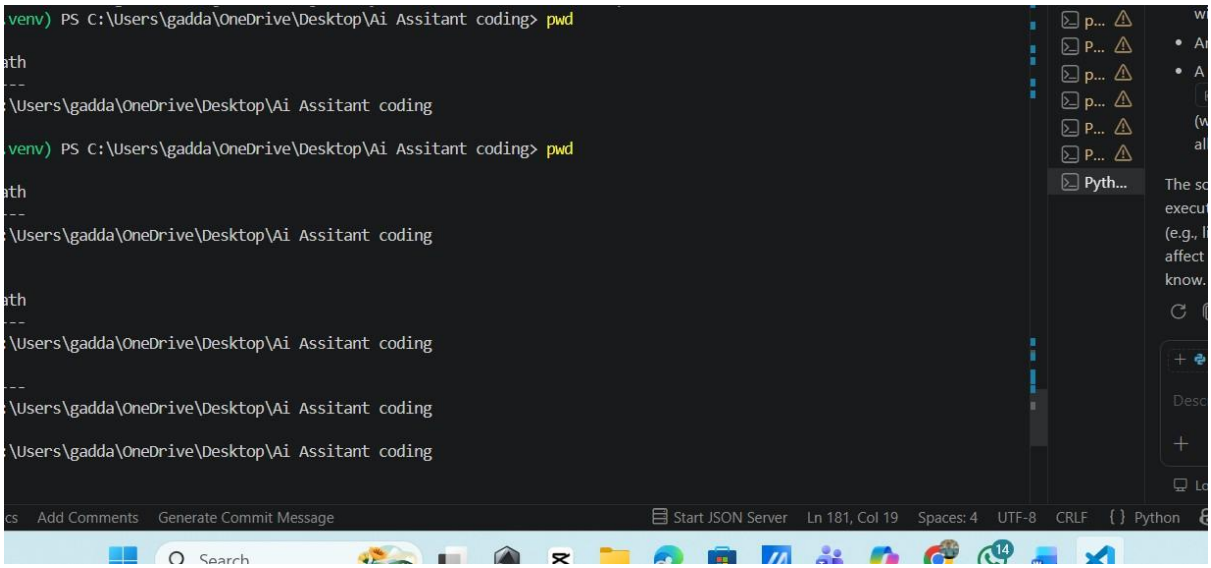
Code:



```
mport subprocess
mport shlex

ef secure_command_execution():
    allowed_commands = ["ls", "pwd", "echo", "cat", "head", "tail"]  # Whitelist of safe commands

    user_input = input("Enter a command (ls, pwd, echo, cat, head, tail): ").strip()

    # Input validation: only allow whitelisted commands, no shell metacharacters
    if not user_input or any(char in user_input for char in ";|&`"):
        print("Invalid or unsafe command.")
        return

    parts = shlex.split(user_input)
    if not parts or parts[0] not in allowed_commands:
        print("Command not allowed.")
        return

    try:
        result = subprocess.run(parts, capture_output=True, text=True, timeout=10)
        print("Output:", result.stdout)
        if result.stderr:
            print("Error:", result.stderr)
    except subprocess.TimeoutExpired:
        print("Command timed out.")
    except Exception as e:
        print(f"Error executing command: {e}")

f __name__ == "__main__":
    secure_command_execution()
```

```
def secure_command_execution():
        print("Command timed out.")
    except Exception as e:
        print(f"Error executing command: {e}")

if __name__ == "__main__":
    secure_command_execution()
```

Output:

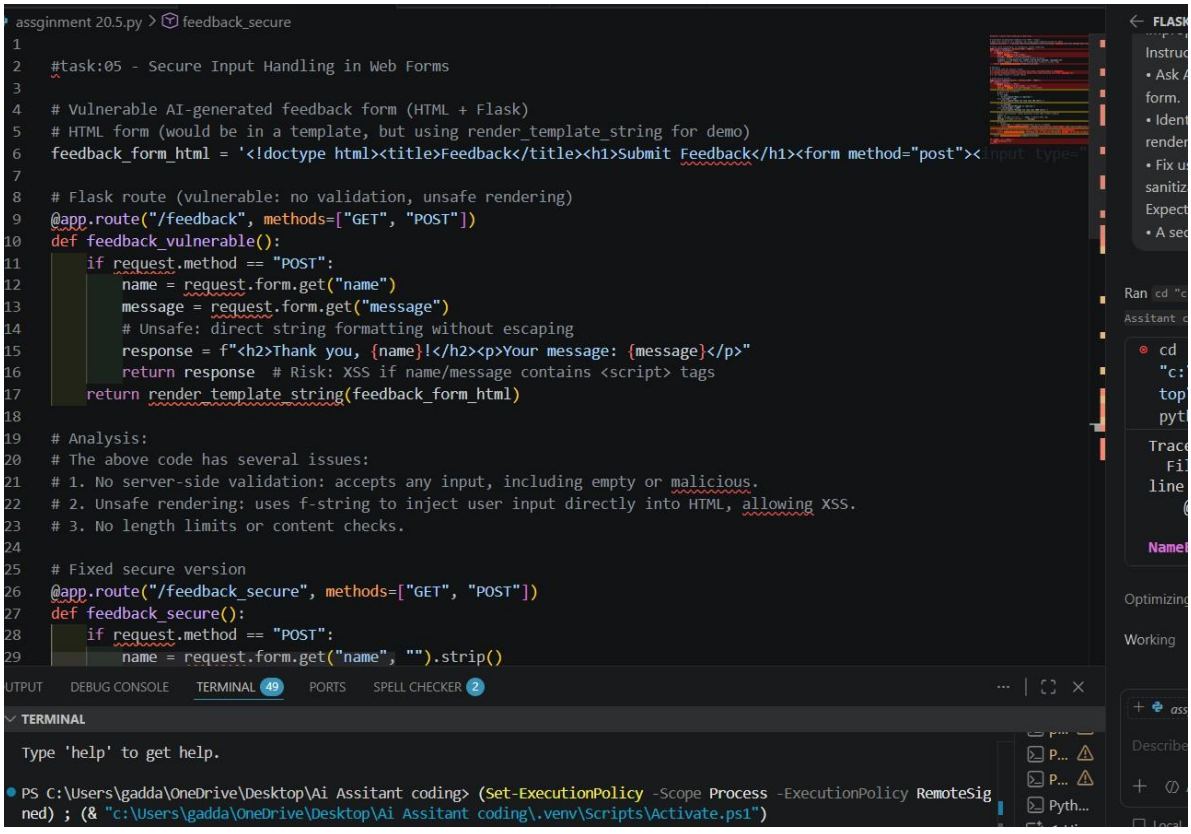


Observation:

- The AI-generated script used functions like os.system() or subprocess with **unsanitized user input**, making it vulnerable to **command injection attacks**.
- Attackers can inject malicious commands (e.g., ; rm -rf /) through input, leading to **data loss or system compromise**.
- The insecure approach directly concatenates user input into system commands without validation.
- The secure version avoids shell execution, uses subprocess.run() with argument lists, and validates input to allow only safe values.

Task:05

```python
@app.route("/feedback_secure", methods=["GET", "POST"])
def feedback_secure():
    if request.method == "POST":
        name = request.form.get("name", "").strip()
        message = request.form.get("message", "").strip()

        # Server-side validation
        errors = []
        if not name:
            errors.append("Name is required.")
        elif len(name) > 100:
            errors.append("Name too long (max 100 chars).")

        if not message:
            errors.append("Message is required.")
        elif len(message) > 1000:
            errors.append("Message too long (max 1000 chars).")

        # Basic sanitization: remove potential script tags (simple example)
        import re
        name = re.sub(r'<[^>]+>', '', name)  # Remove HTML tags
        message = re.sub(r'<[^>]+>', '', message)

        if errors:
            error_html = "".join(f"<li>{e}</li>" for e in errors)
            return render_template_string(f'{feedback_form_html}<ul style="color: red;">{error_html}</ul>

        # Safe rendering using Jinja2 (Flask's render_template_string escapes automatically)
        return render_template_string('<h2>Thank you, {{ name }}!</h2><p>Your message: {{ message }}</p><
```



Output:

Observation

- The AI-generated form accepted user input without proper **server-side validation**, making it vulnerable to attacks like **XSS and injection**.
- User inputs were directly rendered in HTML without sanitization, allowing execution of **malicious scripts**.
- The secure version adds **input validation, escaping, and safe rendering**, ensuring protection against injection attack